Production Order Application Module — Function Design (機能設計)

> **Revision of 2026-10-24 (WI-017).** This file supersedes `001_DD-FN_製造指示登録・編集.md`, which stays unchanged as the earlier approved revision. Change: the production-order notes limit is 1000 characters (REQ-090, supersedes the 500 of REQ-016); see `work-items/WI-017/`.

001_DD-FN — used by 001_DD and 001_DD-API (and later by Screen B's list DD), requirements REQ-010–REQ-018.

Document control (改版履歴)

Field	Value
Document ID	001_DD-FN
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-002
Created by	Claude (for ThanhTN)
Created date	2026-09-18
Last updated by	Claude (for ThongTM)
Last updated date	2026-10-24

Version	Date	Author	Revision content
1	2026-09-18	Claude (for ThanhTN)	Initial creation. Moves 001_DD revision 1's modules 3–5 (service, plant clock, order-number issuer) into this document
2	2026-09-18	Claude (for ThanhTN)	Aligned with implementation (plan revision 2): generated values come back via RETURNING instead of a re-read; the loaded `xmin` is the concurrency original; `AllowedNext()` is an extension method
3	2026-09-22	Claude (for ThanhTN)	WI-004 REQ-033: UpdateAsync step 7 records the completion time through the entity; no new step, no new failure path
4	2026-10-24	Claude (for ThongTM)	WI-017 (REQ-090–REQ-092): notes limit 500 → 1000 characters. `Notes` maximum length 1000 in the order DTO; `ProductionOrder.MaxNotesLength` = 1000. No step or failure path changes. Dated file name per WI-017 DEC-002

Overview and method index

Field	Value
Module name	`ProductionOrderService` plus its supporting services `PlantClock` and `OrderNumberIssuer` (`ProductionManagementAI.Application.ProductionOrders` , implementations of the two interfaces in `ProductionManagementAI.Infrastructure.ProductionOrders`)
Overview	Application-layer use cases for production orders: validate input, apply Domain rules, persist through the repository, and return a typed result the API maps to HTTP responses. The clock and the issuer are shared, so Screen B (list, overdue display) will reuse them

No	Method name	Overview	Notes
1	`ProductionOrderService.ListProductsAsync`	Products for pickers, ordered by SKU	API-PRD-01
2	`ProductionOrderService.GetAsync`	One order as `ProductionOrderResponse`	API-PO-02
3	`ProductionOrderService.CreateAsync`	Validate, issue number, insert	API-PO-01
4	`ProductionOrderService.UpdateAsync`	Validate, version check, Domain rules, save	API-PO-03
5	`ProductionOrderMapper.ToResponse`	Entity → response DTO (value mapping below)	Internal, static
6	`IPlantClock.Today` / `CurrentYear`	Plant-local date and year (`Asia/Tokyo`)	Shared
7	`IOrderNumberIssuer.NextAsync`	Per-year sequence via the counter upsert	Shared

Every service method takes a `CancellationToken` and returns `Result<T>` , a discriminated result with variants `Ok(T)` , `Invalid(errors)` , `NotFound` , `Conflict` , `RuleViolation(code)` . The controller maps these to 200/201, 400, 404, 409 and 422 respectively (DEC-023). Unexpected exceptions aren't turned into a `Result` ; they propagate to the global Problem Details handler (500, MSG-E013).

Shared utility references

No	Name	Overview	Notes
1	`TimeProvider` (.NET)	Injected time source; `FakeTimeProvider` in tests	Registered as `TimeProvider.System`
2	`IProductionOrderRepository`	EF Core data access: `ListProductsAsync` , `ProductExistsAsync` , `FindAsync(id, tracked)` , `Add` , `SaveChangesAsync` , `BeginTransactionAsync`	Infrastructure; single-table queries with column projections (001_DD "Database and transaction mapping")
3	`ProductionOrderTelemetry`	`ActivitySource` + `Meter` named `ProductionManagementAI.ProductionOrders`	001_DD "Observability"
4	`ILogger<ProductionOrderService>`	Structured logging with `LoggerMessage` source-generated methods	Never logs notes text

Request data

Parameters this module sends to the database (through the repository and issuer). All are parameterized; nothing is concatenated into SQL (`ai/rules/database.md`).

No	Name	Variable name	Type	Length	Used by (which method(s))	Notes
1	Order ID	`id`	`Guid`	—	GetAsync, UpdateAsync	Primary-key lookup
2	Product ID	`productId`	`Guid`	—	CreateAsync, UpdateAsync	`ProductExistsAsync`
3	Order year	`year`	`short`	—	CreateAsync → NextAsync	Counter upsert `@year`
4	Original row version	`xmin`	`uint`	—	UpdateAsync	EF adds `WHERE xmin = @original`
5	Entity values	`ProductionOrder`	entity	—	CreateAsync, UpdateAsync	INSERT / UPDATE via EF change tracking

Response / value mapping

`ProductionOrderResponse` (field catalog in 001_DD-API) is built by `ProductionOrderMapper.ToResponse`:

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
1	ID	`Id`	`Guid`	—	yes	`entity.Id`	
2	Order number	`OrderNumber`	`string`	13	yes	`entity.OrderNumber` (DB-generated; EF reads it back via `RETURNING` on insert)	
3	Product ID	`ProductId`	`Guid`	—	yes	as stored	
4	Quantity	`Quantity`	`int`	—	yes	as stored	
5	Due date	`DueDate`	`DateOnly`	—	yes	as stored	Serialized `YYYY-MM-DD`
6	Status	`Status`	`string`	—	yes	`entity.Status.ToString()`	
7	Allowed next statuses	`AllowedNextStatuses`	`string[]`	—	yes	`entity.Status.AllowedNext()` (`ProductionOrderStatusExtensions`), in enum order	M-02
8	Product/quantity editable	`IsProductQuantityEditable`	`bool`	—	yes	`entity.Status == Draft`	REQ-018

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
9	Notes	`Notes`	`string?`	1000	yes	as stored (`null` when empty)	
10	Created at	`CreatedAt`	`DateTimeOffset`	—	yes	`created_at_utc`	UTC
11	Updated at	`UpdatedAt`	`DateTimeOffset`	—	yes	`updated_at_utc`	UTC
12	Version	`Version`	`uint`	—	yes	`entity.RowVersion` (`xmin`)	DEC-014

`ProductResponse` = `{ Id, Sku, Name }`, projected directly in the query.

Method design

1. ProductionOrderService.ListProductsAsync

Field	Value
Description	Returns every product ordered by SKU for the product picker
Return type	`Task<IReadOnlyList<ProductResponse>>`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`CancellationToken`	`ct`	

Return value

Type	Name	Description
`IReadOnlyList<ProductResponse>`	products	30 rows with the current seed; empty list is valid (MSG-E014 on the client)

Processing overview: a single no-tracking projection. No paging (DEC-019, DEC-022).

Processing flow

Step	Description	Calls
1	`SELECT id, sku, name FROM products ORDER BY sku`	`IProductionOrderRepository.ListProductsAsync`
2	Return the list	—

2. ProductionOrderService.GetAsync

Field	Value
Description	Loads one order for the edit screen
Return type	`Task<Result<ProductionOrderResponse>>`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`Guid`	`id`	Order ID from the route
2	`CancellationToken`	`ct`	

Return value

Type	Name	Description
`Result<ProductionOrderResponse>`	result	`Ok` or `NotFound`

Processing overview: a no-tracking load including `xmin`, then mapped.

Processing flow

Step	Description	Calls
1	`FindAsync(id, tracked: false)`	repository
2	Not found → `NotFound`	—
3	`Ok(ToResponse(order))`	`ProductionOrderMapper.ToResponse`

3. ProductionOrderService.CreateAsync

Field	Value
Description	Creates a `Draft` order with the next `PO-YYYY-NNNNN` number (REQ-010)
Return type	`Task<Result<ProductionOrderResponse>>`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`CreateProductionOrderRequest`	`request`	`ProductId?`, `Quantity?`, `DueDate?`, `Notes?` (nullable so a missing field is reported as "required", not a binding error)
2	`CancellationToken`	`ct`	

Return value

Type	Name	Description
`Result<ProductionOrderResponse>`	result	`Ok` (controller returns 201 + `Location`) or `Invalid(errors)`

Processing overview: collect every field error before touching the database. Then run the data-dependent checks. Then issue the number and insert inside one transaction, so a failed insert doesn't consume a number (001_DB).

Processing flow

Step	Description	Calls
1	Start span `ProductionOrder.Create`	`ProductionOrderTelemetry`
2	Required checks: `ProductId` (MSG-E001), `Quantity` (MSG-E003), `DueDate` (MSG-E004)	—
3	Value checks: `ValidateQuantity` (MSG-E003 / MSG-E010), `NormalizeNotes` (trim, empty → null) + `ValidateNotes` (MSG-E006)	`ProductionOrder` static validators (001_DD module 1)
4	Data checks, run only for fields without errors: `ProductExistsAsync` (MSG-E002); `DueDate < IPlantClock.Today` → MSG-E005	repository; `IPlantClock.Today`
5	Any errors → `Invalid(errors)`; counter `created{outcome=validation_failed}`; log `ProductionOrderValidationFailed` (field names only)	—

Step	Description	Calls
6	`BeginTransactionAsync`	repository
7	`seq = NextAsync(IPlantClock.CurrentYear)`	`IOrderNumberIssuer.NextAsync`
8	`order = ProductionOrder.Create(productId, quantity, dueDate, notes, year, seq, utcNow)`	Domain
9	`Add(order)`, `SaveChangesAsync`, commit	repository
10	The generated `order_number` and the new `xmin` are already on the entity: EF Core reads them back via `RETURNING` during `SaveChangesAsync` (no extra query)	EF Core
11	Counter `created{outcome=success}`; span attributes `production_order.id`, `production_order.number`; `Ok(ToResponse(order))`	telemetry; mapper

Failure in steps 7–9 (e.g. counter CHECK violation at the 100,000th order of a year) → roll back, counter `created{outcome=error}`, log Error, rethrow → 500 MSG-E013 (DEC-013).

4. ProductionOrderService.UpdateAsync

Field	Value
Description	Saves edits and status changes with stale-save protection (REQ-011, REQ-017, REQ-018, DEC-010)
Return type	`Task<Result<ProductionOrderResponse>>`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`Guid`	`id`	Route ID
2	`UpdateProductionOrderRequest`	`request`	`ProductId?`, `Quantity?`, `DueDate?`, `Status?`, `Notes?`, `Version?`
3	`CancellationToken`	`ct`	

Return value

Type	Name	Description
`Result<ProductionOrderResponse>`	result	`Ok`, `Invalid`, `NotFound`, `Conflict`, `RuleViolation("MSG-E007" | "MSG-E008")`

Processing overview: the check order is fixed, so each failure has exactly one outcome: shape (400) → existence (404) → version (409) → Domain rules (422) → data-dependent checks (400) → save race (409). Nothing is written unless every check passes (single `SaveChangesAsync`, DEC-007).

Processing flow

Step	Description	Calls
1	Start span `ProductionOrder.Update`	telemetry
2	Required/value checks as in Create steps 2–3, plus `Status` required and a known enum value (MSG-E007 when unknown) and `Version` required (MSG-E009)	Domain validators
3	Errors → `Invalid` (`updated{outcome=validation_failed}`)	—
4	`FindAsync(id, tracked: true)`; missing → `NotFound` (`updated{outcome=not_found}`)	repository
5	`request.Version != order.RowVersion` → `Conflict` (`updated{outcome=conflict}`, log `ProductionOrderConcurrencyConflict`)	—
6	Remember `fromStatus`, `dueDateChanged = request.DueDate != order.DueDate`, `productChanged = request.ProductId != order.ProductId`	—
7	`order.Update(productId, quantity, dueDate, notes, status, utcNow)`; catch `DomainRuleViolation(code)` → `RuleViolation(code)` (`updated{outcome=rule_violation}`, log `ProductionOrderRuleViolated`). The entity is unchanged when it throws. On `InProgress → Completed` the entity also sets `CompletedAtUtc = utcNow` (001_DD module 1 step 5, WI-004 REQ-033); if a later step fails, nothing is saved, so no completion time is recorded	Domain (001_DD module 1)
8	If `productChanged`: `ProductExistsAsync` (MSG-E002). If `dueDateChanged`: `DueDate < IPlantClock.Today` → MSG-E005 (DEC-009). Errors → `Invalid`; the tracked changes are discarded (the context is request-scoped and not saved)	repository; `IPlantClock`
9	`SaveChangesAsync`. EF adds `WHERE xmin = @original`, where the original is the `xmin` loaded in step 4 (already checked equal to `request.Version` in step 5), so a write between steps 4 and 9 is still caught	repository
10	`DbUpdateConcurrencyException` → `Conflict`	—
11	If `status != fromStatus`: counter `status_transitions{from,to}`; span `status.from` / `status.to`	telemetry
12	The new `xmin` comes back via `RETURNING` → `Ok(ToResponse(order))`; `updated{outcome=success}`	mapper

5. ProductionOrderMapper.ToResponse

Field	Value
Description	Pure mapping from entity to `ProductionOrderResponse`

Field	Value
Return type	`ProductionOrderResponse`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`ProductionOrder`	`order`	Loaded entity

Return value

Type	Name	Description
`ProductionOrderResponse`	response	See "Response / value mapping"

Processing overview: field-by-field copy plus the two computed fields. No I/O.

Processing flow

Step	Description	Calls
1	Copy stored fields	—
2	Compute `AllowedNextStatuses` and `IsProductQuantityEditable`	`ProductionOrderStatusExtensions.AllowedNext`

6. IPlantClock.Today / CurrentYear

Field	Value
Description	Plant-local date and year in the timezone configured at `Plant:TimeZone` (default `Asia/Tokyo`, DEC-011, DEC-017)
Return type	`DateOnly` / `short`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments: none. Constructor dependencies: `TimeProvider`, `IOptions<PlantOptions>`.

Return value

Type	Name	Description
`DateOnly`	`Today`	`DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(timeProvider.GetUtcNow(), zone).DateTime)`
`short`	`CurrentYear`	`(short)Today.Year`

Processing overview: `TimeZoneInfo.FindSystemTimeZoneById(options.TimeZone)` is resolved once at construction; IANA IDs work on both Linux and Windows under .NET's ICU support. Options validation at startup rejects an unknown ID, so the app fails fast instead of per request.

Processing flow

Step	Description	Calls
1	Read the current UTC time	`TimeProvider.GetUtcNow`
2	Convert to the plant zone; take the date / year	`TimeZoneInfo.ConvertTime`

7. IOrderNumberIssuer.NextAsync

Field	Value
Description	Returns the next `order_seq` for a year using 001_DB's counter upsert, inside the caller's transaction (DEC-013)
Return type	`Task<int>`
Created by / date	Claude / 2026-09-18
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`short`	`year`	Plant year
2	`CancellationToken`	`ct`	

Return value

Type	Name	Description
`int`	`seq`	1–99999

Processing overview: `Database.SqlQuery<int>($"INSERT ... ON CONFLICT (order_year) DO UPDATE SET last_seq = ... + 1 RETURNING last_seq")`. EF converts the interpolated SQL into parameters. The row lock is held until the caller commits, which serializes same-year creates.

Processing flow

Step	Description	Calls
1	Throw `InvalidOperationException` if `Database.CurrentTransaction` is null (guards against numbers being issued outside the insert's transaction)	—
2	Run the upsert, return `last_seq`	001_DB `production_order_number_counters`
3	A CHECK violation (`last_seq > 99999`) surfaces as `PostgresException` 23514 → caller rolls back → 500	—

Unresolved decisions

None. Related decisions: DEC-009, DEC-010, DEC-013, DEC-014, DEC-017, DEC-023 in `work-items/WI-002/decisions.md`.